

# Docendo

Learning by teaching an AI classmate, and then watching it use what you taught

Team Andrii Negrub, Phil Gustke, Rafael Cruz

Prototype [docendo-jade.vercel.app](https://docendo-jade.vercel.app)

Access code `bandits2026`

Date 8 October 2026

Code [github.com/OLuminaX0/docendo](https://github.com/OLuminaX0/docendo)

## 1 Topic and audience

Our lesson is about multi-armed bandits. A bandit problem is any repeated choice between options whose average payoff you don't know: which restaurant to eat at, which ad to show, which slot machine to play. Every choice pays something and also tells you a little about that option, so there is a constant pull between *exploiting* the option that looks best and *exploring* the others in case one of them is better. We chose it because it fits into half an hour, has a clear structure to teach, and has a misconception worth catching: people tend to trust the option that looked best after a few tries.

Our audience, and our study participants, are EPFL bachelor's and master's students in engineering and computer science. They know what an average and a probability are, but have not seen bandits or reinforcement learning before.

## 2 Learning goal

Our first draft had three goals: compute the regret of a strategy, compare  $\epsilon$ -greedy and UCB, and model a new problem as a bandit. Covering all three needed fifteen minutes of video and twenty facts to teach, far too much for a 30–40 minute session, and the TA advised us to focus on one goal. We kept the part of the topic that the three classic strategies have in common. Learners have to apply each rule to situations they have not seen and explain how the rules differ:

### LEARNING GOAL

Given what has happened so far, predict which option **greedy**,  **$\epsilon$ -greedy** and **UCB** will pick next, and explain why greedy can get stuck on a worse option while  $\epsilon$ -greedy and UCB keep exploring, each in its own way.

| PART               | A LEARNER CAN...                                                                                                                      | TEST ITEMS |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------|------------|
| Greedy and lock-in | say which option greedy picks from the averages so far, and why one unlucky first try can hide the best option for good               | q03–q04    |
| $\epsilon$ -greedy | work out how often $\epsilon$ -greedy shows its current favourite, and explain that its random picks ignore everything it has learned | q05–q07    |
| UCB                | pick the option with the highest average plus bonus, and explain why the bonus, and with it the exploring, shrinks with every try     | q08–q10    |

### 3 The lesson

A session runs in four chapters and takes about 35 minutes. Only the third chapter differs between the two conditions of our study.

| CHAPTER      | TIME     | WHAT THE LEARNER DOES                                                                                                              |
|--------------|----------|------------------------------------------------------------------------------------------------------------------------------------|
| 1 Watch      | 8 min    | Watches one lesson made of three parts of two YouTube videos (ritvikmath and DataMListic), played back to back in a single player. |
| 2 Teach Kai  | ≈ 9 min  | Explains the lesson to Kai, an AI classmate who missed it, until Kai feels ready.                                                  |
| 3 Practice   | 8 min    | Works on eight problems with right/wrong feedback and worked solutions. <b>This is where the conditions differ</b> (Section 5).    |
| 4 Final test | ≈ 10 min | Answers ten multiple-choice questions set in new situations. Graded on the server.                                                 |

### 4 The main activity: teaching Kai

The heart of the lesson is the conversation with Kai (Figure 2, left). Kai is a language model, but it is not allowed to know the topic. Behind the chat, the lesson is stored as a small knowledge graph of nine ideas and thirteen facts. After each message a second model, the judge, decides which facts the learner just explained and has to quote the learner's own words as evidence; a verdict whose quote is not in the learner's message is thrown away. The quotes go into Kai's notebook. Kai's replies are written from that notebook and the conversation only, and a filter blocks lesson vocabulary the learner has not used yet. So Kai knows what it was told, in the learner's words, mistakes included.

Kai is not a passive listener. Every reply ends with one concrete question about the next fact it is still missing ("Do I know in advance how good each restaurant is?"). When an idea is complete, Kai says what it now understands; later it offers a common misconception as its own guess, which the learner has to correct. If the learner is stuck, a button replays the right moment of the video, but only after they have tried to answer. Kai feels ready once 80% of the facts have been explained, or 50% after eight minutes of teaching.

Teaching is a strong learning activity in its own right. Students who prepare to teach and then explain to someone else work harder on the material than students who learn for themselves, the so-called protégé effect (Chase et al., 2009). The gain depends on what kind of explaining happens: tutors learn most when their pupil's questions make them repair and extend their explanations (knowledge-building) rather than recite what they know (knowledge-telling; Roscoe & Chi, 2007). Kai's targeted questions are meant to push learners into the first kind. In terms of the ICAP framework (Chi & Wylie, 2014) this makes teaching Kai an interactive activity, not just a constructive one. Keeping Kai ignorant matters too. Jin et al. (2024) found that learners stop really teaching when the AI tutee already seems to know the answers.

## 5 What the AI adds: recursive feedback

Teachers learn not only from teaching but from seeing what their students do with it afterwards. Okita and Schwartz (2013) call this *recursive feedback*. In their second experiment, high-school students taught logic rules to a computer agent, Moby, and then watched it play a prediction game in which every move was marked right or wrong. A control group played the same game themselves and received the same marks. On a later test the students who had taught Moby did better ( $d = 0.75$ ), although both groups saw the same feedback for the same amount of time. Our study is a conceptual replication of that experiment with Kai.

### The experimental condition

After teaching, Kai solves the practice problems itself, step by step, using only its notebook. Each step of a correct solution relies on a specific fact from the lesson. If the learner never taught that fact, or taught it wrongly, Kai's step goes wrong in the same way. A light shows whether Kai's final answer is right. A red light says only that something is wrong, not where. The learner clicks the sentence they think is faulty, and Kai shows which of the learner's own statements it was relying on. The learner can then correct that point in the chat and let Kai try again. Figure 1 shows the idea on a problem from our practice set, with a misconception we expect to be common: mixing up how  $\epsilon$ -greedy and UCB explore.

**Problem 3: an explore day.** You use  $\epsilon$ -greedy with  $\epsilon = 0.1$ , and today is an explore day. Your averages so far: truck A 5.5 (tried 20 times), B 4.0 (18 times), C 1.2 (15 times). Where can you end up eating?



#### Kai's solution

1. Today is an explore day, so I don't just go to the truck with the best average.
2. When I explore, I go to the truck I have tried the least, which is C.
3. So today I eat at C, and only at C.



#### Kai's answer is wrong

Click the step where Kai goes wrong.

I got step 2 from you. You said: "when it explores it goes for the restaurant it has visited the least".

**Figure 1.** Design mockup of the recursive-feedback practice (not built yet). Kai's step 2 comes from something the learner taught. In fact,  $\epsilon$ -greedy explores completely at random; aiming at the least-tried option is closer to what UCB does. Finding the step makes the learner notice the confusion in their own explanation.

Only an AI tutee can offer this. In Okita and Schwartz's first experiment the "pupil" was a recording, the same for every participant, and Moby could only be taught through a formal rule language. A language model can take whatever a learner actually said and apply it to new problems. It is also why we restrict Kai so strictly: if it solved problems from its own background knowledge, its mistakes would no longer come from the teaching.

There are several reasons to expect this to help. Seeing their explanation used on a new problem lets learners find gaps they could not see while explaining, and fix them in their own understanding as well as Kai's. This is the production-appropriation cycle described by Okita and Schwartz. The feedback is also about the learner's reasoning, not just about a single answer, which is the level at

which feedback tends to work best (Hattie & Timperley, 2007). Because the mistake is Kai's, learners can look at it without feeling judged; Chase et al. (2009) describe this as an ego-protective buffer. Finally, spotting errors in someone else's reasoning is easier than catching your own while you solve a problem, and practising it improves self-correction (Okita, 2014).

## 6 The control condition: direct feedback

In the control condition everything stays the same except one thing: the learner solves the practice problems themselves (Figure 2, right). They get the same eight problems in the same eight minutes, and the same light, which they can check as often as they like. A space next to each problem lets them write down their working, much like the control group in the original study, which also had to write down its rules. Two levels of help are available. The first replays the moment of the video that explains what the problem needs; the second shows the full worked solution, the same solution the experimental group can open.

We made the control strong on purpose. Practice with immediate feedback and worked solutions is a good way to learn, so if the experimental group does better, it cannot be because it saw more content, more solutions or spent more time. As in the original experiment, the only difference is whose answers the feedback is about.

The figure consists of two side-by-side screenshots of the Docendo interface. The left screenshot shows a teaching session with Kai, featuring a chat interface with blue and grey bubbles and a progress bar at 19%. The right screenshot shows a practice session with a 'pick and why' question about a greedy algorithm, multiple-choice options A-D, and a feedback bar with 'Check', 'Correct', 'Rewatch', and 'Show solution' buttons.

**Figure 2.** The prototype. *Left:* teaching Kai, a real exchange with the live model. When the learner asks to move on, Kai first asks for the one fact still missing; once the idea is complete, it confirms what it understood and offers a misconception to be corrected. *Right:* practice in the control condition: the learner's working, a "pick and why" question, the right/wrong light, and the two levels of help.

## 7 Evaluation plan

**Procedure.** Participants will open the study link, read what is recorded and start. The server will assign each session to a condition at random. The whole session runs in the browser in about 35 minutes, without accounts or names: each session gets a random id, and an optional participant code in the link lets us match it with other data, such as a questionnaire.

**Hypothesis and test.** We expect learners in the recursive-feedback condition to score higher on the final test than learners in the direct-feedback condition. The test has ten multiple-choice items, one or more for each part of the goal. None of them repeats a practice problem. Each tests the same principle in a different setting: banner ads, office coffee machines, a study app and podcasts, where practice used food trucks, slot machines and headlines. Most items ask the learner to pick an option *and* the right reason for it, so understanding counts, not only the final pick.

**Process data.** The prototype logs every step of a session, from each message to Kai and the facts the judge credited to every practice check, opened solution and test answer. Kai's understanding at the end of teaching measures how well someone taught. It is recorded before the conditions start to differ, so we can use it as a covariate.

**Manipulation checks.** The first check, suggested by the TA, is leakage: after the sessions, an independent model will check every reply and solution step Kai produced for lesson content that was not in its notebook, and we will review a sample by hand. If Kai leaks more than rarely, its mistakes no longer reflect the teaching. We will also check that Kai's wrong steps really trace back to gaps or errors in what the learner said, and that both groups spent the same time in practice and opened a similar number of worked solutions.

**Analysis and power.** We will compare test scores with an analysis of covariance (condition, plus teaching quality as covariate) and report the effect size with its confidence interval. To detect the original effect of  $d = 0.75$  with 80% power, we would need about 29 participants per condition, or 22 with a one-sided test. With the 15 to 20 per condition we can realistically recruit, power lies between about 0.55 and 0.75, depending on the final sample and on whether we test one-sided. Our setting also differs from the original (university students, 35 minutes instead of 75, a language model instead of a rule-based agent), so the true effect may be smaller. We will report effect sizes, not only significance, and use the process data to explain what we find.

## 8 Status and next steps

The prototype at [docendo-jade.vercel.app](https://docendo-jade.vercel.app) (access code `bandits2026`) runs the complete direct-feedback condition, from the video to the test results, with logging switched on. Learners can also open "Kai's mind", which shows which ideas Kai has understood without revealing what it still lacks. The next step is the recursive-feedback practice of Figure 1, followed by a pilot with two or three people.

Two design questions are still open, and the pilot will settle them. The first is how much Kai should ask while being taught. The pupil in the original first experiment only acknowledged what she heard; our Kai asks a targeted question after every message, which is closer to good tutoring but further from the original. We want to see both with real learners before deciding, as agreed with the TA, and the chosen version will be the same in both conditions. The second is Kai's readiness rule, which we will tune to keep the session close to 35 minutes.

## References

- Chase, C. C., Chin, D. B., Oppezzo, M. A., & Schwartz, D. L. (2009). Teachable agents and the protégé effect: Increasing the effort towards learning. *Journal of Science Education and Technology*, 18(4), 334–352.
- Chi, M. T. H., & Wylie, R. (2014). The ICAP framework: Linking cognitive engagement to active learning outcomes. *Educational Psychologist*, 49(4), 219–243.
- Hattie, J., & Timperley, H. (2007). The power of feedback. *Review of Educational Research*, 77(1), 81–112.
- Jin, H., Lee, S., Shin, H., & Kim, J. (2024). Teach AI how to code: Using large language models as teachable agents for programming education. In *Proceedings of CHI 2024*.
- Okita, S. Y. (2014). Learning from the folly of others: Learning to self-correct by monitoring the reasoning of virtual characters in a computer-supported mathematics learning environment. *Computers & Education*, 71, 257–278.
- Okita, S. Y., & Schwartz, D. L. (2013). Learning by teaching human pupils and teachable agents: The importance of recursive feedback. *Journal of the Learning Sciences*, 22(3), 375–412.
- Roscoe, R. D., & Chi, M. T. H. (2007). Understanding tutor learning: Knowledge-building and knowledge-telling in peer tutors' explanations and questions. *Review of Educational Research*, 77(4), 534–574.

---

**Use of AI.** We used an AI coding assistant (Claude Code) to build the prototype and to help draft this report. Kai runs on Gemini 2.5 Flash. The design decisions are ours, and we reviewed everything before submitting. Each team member's AI interaction logs are attached to the submission.